Deep Learning Models Interview Cheatsheet

Prepared for

Saim Malik

Interview with Kevin DiGilio — CTO, Akuret Solutions

AI-Powered Shelf Intelligence for Grocery Retail

Models Covered	CNN, VGG16, VGG19, ResNet50, YOLO, LSTM
Key Topics	Computer Vision, Object Detection, Transfer Learning, MLOps
Your Projects	YOLO Pipe Detection, Movie Classifier, Neighborhoods.com
Interview Focus	CV pipeline, Data engineering, SaaS architecture, MLOps

Table of Contents

5 questions on supervised learning, overfitting, embeddings

18. Smart Questions to Ask Kevin

6 ready-to-use questions that show depth

1. CNNs — The Foundation (Know This Cold)

A CNN is a neural network designed specifically for visual data. Instead of treating an image as a flat list of pixels, it uses **convolutional filters** (small sliding windows) that scan across the image to detect patterns — edges first, then textures, then shapes, then objects.

Akuret's shelf scanning uses CNNs (or CNN-based architectures) to detect products, empty shelves, misplaced items, and planogram violations from smartphone video frames.

Key Concepts

Convolutional Layer: A set of learnable filters (e.g., 3x3 or 5x5) that slide across the input image. Each filter detects a specific feature. The output is a 'feature map.' Early layers detect edges; deeper layers detect complex patterns like product labels.

Max Pooling: Like stepping back from a poster and squinting — you lose tiny details but still see the big picture. Takes a 2x2 window, keeps only the highest value. A 100x100 feature map becomes 50x50. Benefits: faster computation, **translation invariance** (a Coke can shifted 2 pixels still gets detected), and preserves strongest signals.

> ***Interview line:*** *"Max pooling shrinks spatial dimensions while preserving the most important features. It also makes the model robust to small positional shifts — which matters for shelf scanning where products aren't pixel-perfect aligned."*

Fully Connected (Dense) Layer: The conv layers are **detectives** collecting evidence. The FC layer is the **judge** making the final call. The catch: FC layers are parameter-heavy. In VGG16, conv layers have ~15M params but the three FC layers have ~102M (7x more). That's why ResNet replaced them with Global Average Pooling.

> ***Interview line:*** *"Conv layers extract spatial features, FC layers learn the mapping to class labels. They're parameter-heavy, which is why ResNet moved to Global Average Pooling to stay lean."*

ReLU (Rectified Linear Unit): Positive → pass through, negative → zero. max(0, x). Like a **bouncer** — useful signals get in, noise gets turned away. Without activation functions, 100 layers = one big linear equation. ReLU adds **non-linearity**. Its gradient is 0 or 1 — no vanishing gradients.

> ***Interview line:*** *"ReLU adds non-linearity so deep networks can learn complex patterns, and its gradient doesn't shrink during backprop like sigmoid. Computationally trivial — just zeroing out negatives."*

Softmax: Converts raw scores into probabilities summing to 100%. Raw: Good=5.2, Normal=2.1, Bad=0.7 → Softmax: Good 85%, Normal 12%, Bad 3%. Tells you not just **what** the model predicts but **how sure** it is. At Akuret, low-confidence detections might need human review.

> ***Interview line:*** *"Softmax normalizes raw outputs into a probability distribution. It tells you not just what the model predicts, but how sure it is — which matters operationally for deciding whether to trust a detection or flag it for review."*

Batch Normalization: Like resetting baton speed between relay runners. Normalizes inputs to each layer — centering around zero with unit variance. Benefits: faster convergence, mild regularization, less sensitivity to initialization. This is why ResNet can go 50+ layers.

> ***Interview line:*** *"BatchNorm stabilizes and accelerates training by keeping each layer's inputs in a well-behaved range. Essential for deep networks — ResNet has it in every block."*

Dropout: Like a soccer team forced to play without their star — the whole team gets stronger. Randomly turns off 50% of neurons each training step, forcing **redundant representations**. Your FYP needed this: ~200K frames but VGG has 138M parameters = massive overfitting risk.

> ***Interview line:*** *"Dropout forces redundant feature representations instead of depending on specific neurons. Essential for preventing overfitting when your dataset is small relative to model capacity."*

If Kevin asks: 'Explain how a CNN processes a shelf image'

> *"The image enters as a matrix of pixel values — RGB channels. The first convolutional layers detect low-level features like edges. Deeper layers combine those into higher-level features — product shapes, label text patterns. Pooling layers reduce spatial size. Finally, the fully connected layers classify the image. The key insight is that the convolutional filters are learned, not hand-coded — the network discovers what features matter from training data."*

2. VGG16 & VGG19 — Your FYP Models

VGG (Visual Geometry Group, Oxford, 2014) proved that **depth matters** — stacking many small 3x3 convolutional filters outperforms fewer larger filters. VGG16 has 16 weight layers; VGG19 has 19.

Architecture (VGG16)

```
Input (224x224x3) → [Conv3x3, 64] x2 → MaxPool → [Conv3x3, 128] x2 → MaxPool
→ [Conv3x3, 256] x3 → MaxPool → [Conv3x3, 512] x3 → MaxPool
→ [Conv3x3, 512] x3 → MaxPool → FC-4096 → FC-4096 → FC-classes → Softmax
```

Key Design Decisions

All 3x3 filters: Two stacked 3x3 convs = same receptive field as one 5x5, but fewer parameters and more non-linearity.

Double channels after each pool: 64 → 128 → 256 → 512 → 512. More filters = more complex features.

138 million parameters: Most are in the FC layers (~102M of 138M = 74%). This is the key weakness.

If Kevin asks: 'Why did you choose VGG for your project?'

> *"VGG was a deliberate choice as a strong baseline. Its uniform architecture makes it interpretable and easy to compare configurations. We tested VGG16 in two configurations and VGG19 to see how additional depth affected performance. The main limitation was overfitting risk given our dataset size, which is why we also tested ResNet50."*

3. ResNet50 — Your FYP Model

Before ResNet, deeper networks eventually **hurt** performance due to the **vanishing gradient problem**: gradients shrink during backpropagation so early layers stop learning. VGG maxes out around 19 layers.

The Breakthrough: Skip Connections

Instead of learning H(x) directly, each block learns the **residual** F(x) = H(x) - x, then adds back the input: **Output = F(x) + x**. If optimal = do nothing, learning F(x) ≈ 0 is easy. Gradients flow directly through the skip connection — 50, 100, or 152 layers train effectively.

Key Numbers

Metric	ResNet50	VGG16
Parameters	~25.6M	~138M (5x more)
FC layer overhead	Global Avg Pool (minimal)	~102M (74% of total)
Max effective depth	50-152 layers	~19 layers
Skip connections	Yes (every block)	No
Batch normalization	Every block	Not used

If Kevin asks: 'What's a skip connection?'

"A skip connection lets the input bypass the block's transformations and get added directly to its output. The block only learns the difference — the residual. This solves vanishing gradients because during backpropagation, gradients flow directly through the skip path, even in very deep networks. It's why ResNet goes to 50+ layers while VGG struggles past 19."

4. YOLO — You Only Look Once

YOLO is designed for **real-time object detection** — finding what objects are in an image AND where they are, in a single pass. This is the most likely architecture for Akuret's shelf scanning.

Classification vs Detection — Critical Distinction

	Classification (Your FYP)	Detection (Akuret needs)
Question	"What is this image?"	"What objects are here and where?"
Output	One label: Good 85%	Many boxes: Coke at (x,y), gap at (x2,y2)
Model	ResNet50, VGG16/19	YOLO, Faster R-CNN, RT-DETR
Architecture	CNN → Global Avg Pool → FC → Softmax	Backbone → Neck (FPN) → Detection Head → NMS

How YOLO Works (3 Steps)

Step 1 — Grid: Divide the image into an SxS grid (e.g., 13x13). Each cell is responsible for detecting objects whose center falls within it.

Step 2 — Predict: Each grid cell predicts B bounding boxes (x, y, w, h), confidence scores, and C class probabilities — all in one forward pass.

Step 3 — Filter (NMS): Many overlapping boxes detect the same object. Non-Maximum Suppression keeps the highest-confidence box and removes duplicates (IoU > 0.5 = same object).

YOLO Versions

Version	Year	Key Innovation
YOLOv1	2016	Original single-pass detector
YOLOv3	2018	Darknet-53 backbone, 3-scale detection, FPN
YOLOv5	2020	PyTorch (Ultralytics), easy training, multiple sizes (s/m/l/x)
YOLOv8	2023	Anchor-free, decoupled head, detection + segmentation

Key YOLO Concepts

Anchor Boxes: Pre-defined box shapes the network adjusts. Learned via k-means clustering. YOLOv8 is anchor-free.

IoU (Intersection over Union): Overlap area / Total area. Used for NMS and evaluation.

FPN (Feature Pyramid Network): Multi-scale detection — combines shallow (small objects) and deep (large objects) features.

mAP (mean Average Precision): Standard metric. mAP@0.5 = IoU threshold 50%.

If Kevin asks: 'How would YOLO apply to shelf scanning?'

"Shelf scanning is a textbook YOLO use case. You detect multiple objects — every product, gap, price tag — in a single image, in near real-time. YOLO does this in one forward pass. The trained model outputs all detected objects with positions and confidence scores, then you compare against the planogram to flag discrepancies. The challenges: products look very similar (different flavors), occlusion, varying lighting, and new products the model hasn't seen."

5. LSTM — Long Short-Term Memory

LSTM is a type of Recurrent Neural Network (RNN) for **sequential data** — time series, text, video frames. Regular RNNs forget long-range dependencies; LSTMs solve this with a gating mechanism.

Three gates: Forget Gate (discard irrelevant info), Input Gate (store new info), Output Gate (decide what to output). These control what the network remembers across time steps.

CNN + LSTM for video: CNN extracts spatial features per frame → LSTM processes the sequence of feature vectors → classification based on temporal patterns.

Relevance to Akuret: They process video scans, not single images. Temporal models could help track shelf sections, handle blurry transitions, or reconstruct complete shelf state from partial views.

---

6. Transfer Learning

Instead of training from scratch, start with a model pre-trained on ImageNet (1.4M images, 1000 classes) and adapt it to your task. Early CNN layers learn universal features (edges, textures) that transfer across tasks.

Feature Extraction: Freeze all conv layers, only train new FC layers. Fast, works with small datasets.

Fine-tuning: Freeze early layers, unfreeze later layers. Better accuracy, needs more data.

Your FYP: Almost certainly used pre-trained ImageNet weights and fine-tuned later layers. Training VGG16 (138M params) from scratch on your dataset would overfit badly.

> *"Transfer learning is probably central to how Akuret handles new retail chains. You can't label millions of shelf images per client. Instead, fine-tune a pre-trained model on that client's products and conditions. The labeled data requirement drops dramatically."*

---

7. Key Metrics

Metric	What it measures	Shelf scanning example
Accuracy	Correct / Total predictions	Misleading on imbalanced data
Precision	Of predicted X, how many were actually X?	"When we say empty, it really is empty"
Recall	Of actual X, how many did we find?	"We catch almost every out-of-stock"
F1 Score	Harmonic mean of precision & recall	Best for imbalanced classes
mAP	Averaged precision across classes + IoU	Standard for detection (YOLO)

Loss Functions: Classification uses cross-entropy. Detection uses composite loss = classification loss + localization loss (box coordinates) + confidence loss.

8. Quick-Fire Answers

Q: What's the difference between detection and classification?

Classification = one label per image. Detection = multiple objects with bounding boxes. Classification asks 'what is this?' Detection asks 'what's here and where?'

Q: What's overfitting and how do you prevent it?

Model memorizes training data instead of learning general patterns. Prevention: dropout, data augmentation, early stopping, regularization, transfer learning, and more diverse training data.

Q: What's data augmentation?

Artificially expanding training data with transforms — random crops, flips, rotation, brightness changes. For shelves: different lighting, angles, perspective shifts.

Q: What's the vanishing gradient problem?

Gradients shrink exponentially through many layers — early layers stop learning. ResNet's skip connections solve this. BatchNorm also helps.

Q: What's Global Average Pooling?

Instead of flattening into huge FC layers (like VGG — 102M params), take one average per feature map. Dramatically fewer params, less overfitting. ResNet uses this; VGG doesn't.

Q: Why not just use the biggest model?

Trade-off: accuracy vs speed vs cost. For Akuret, store associates won't wait 30 seconds. Use smaller YOLO variant, or model distillation (compress large model's knowledge into a smaller one).

---

9. How Models Connect to Akuret's Stack

```
Phone (30-sec scan) → Frame Extraction → YOLO Detection
→ Planogram Comparison → Gap Detection (OOS, misplaced, wrong, low stock)
→ Business Logic (lost $ estimation, priority ranking)
→ Task List → Route to: Store Associate / Warehouse / DSD Vendor
```

Your Experience Maps Here

Akuret Layer	Your Experience
Detection model (YOLO)	YOLO pipe detection + ResNet/VGG FYP
Frame extraction + data pipeline	Neighborhoods.com — 100+ feeds, ETL, Celery
Multi-tenant platform	ParkGuard — RBAC, hierarchical tenants
Gen AI / NLP task generation	LLM pipelines, RAG, LangChain/LangGraph
MLOps / deployment	SageMaker, FastAPI, Docker, CI/CD

10. ResNet50 vs YOLO — The Critical Distinction

	ResNet50	YOLO
Task	Classification — one label per image	Detection — multiple objects + locations
Output	Probability vector: [0.85, 0.12, 0.03]	List: [{class, x, y, w, h, conf}, ...]
Structure	50 conv layers + GAP + FC	Backbone + Neck (FPN) + Detection Head
Parameters	~25.6M	~7M (small) to ~69M (xlarge)
Speed	~5-10ms	~5-20ms
Loss	Cross-entropy only	Class + box position + confidence
Training data	image → one label	image → list of annotated bounding boxes

One-liner: *"ResNet50 tells you what an image is. YOLO tells you what's in an image and where. They're complementary — ResNet can serve as YOLO's backbone. My FYP gave me the backbone knowledge; my YOLO project gave me the detection pipeline."*

11. One-Line Summaries — Rapid Recall

Model	One-Liner
CNN	Learnable sliding filters to extract visual features hierarchically
VGG16	16-layer CNN, all 3x3 convs, simple but huge (138M params)
VGG19	VGG16 + 3 more conv layers — slightly better, even slower
ResNet50	50 layers with skip connections — solves vanishing gradients, 5x fewer params
YOLO	Single-pass object detector — all objects + locations in one forward pass
LSTM	Recurrent network with memory gates for sequential data
Transfer Learning	Start from ImageNet weights, fine-tune for your task
FPN	Multi-scale pyramid — detects small and large objects
NMS	Removes duplicate overlapping detections, keeps the best
mAP	Standard detection metric — averaged precision across classes and IoU

12. Q&A;: YOLO — Fuel Decanting Project

Q: Tell me about your YOLO project

We built a safety monitoring system for fuel decanting at petrol stations. When a tanker arrives to refuel underground storage, a pipe connects the truck to the ground port. We used YOLO to automatically detect the pipe in site images and verify the decanting was happening correctly — automating a safety compliance check.

Q: Why did you choose YOLO over other architectures?

Three reasons. Speed — single forward pass for near-real-time detection. Simplicity — single-stage architecture, easier to train and deploy than two-stage detectors like Faster R-CNN. Ecosystem — Ultralytics YOLOv5 gave us great tooling for custom dataset training, data augmentation, and model export.

Q: What version of YOLO did you use?

YOLOv5. Most mature PyTorch implementation at the time, excellent documentation, multiple model sizes (s/m/l/x). The 's' variant gave us a good speed-accuracy trade-off for our use case.

Q: How did you prepare your training dataset?

Collected images from petrol station sites under varying conditions — different times of day, weather, camera angles, decanting stages. Manually annotated bounding boxes around the pipe using a labeling tool. Applied data augmentation — flips, brightness/contrast, rotation, perspective shifts — to handle real-world variation.

Q: What challenges did you face?

Four main ones: (1) Environmental variation — day/night lighting, weather. Addressed with heavy augmentation. (2) Similar-looking objects — model confused pipe with railings and hoses. Added negative examples and hard-mined false positives. (3) Small object detection — pipe could be small fraction of image. FPN multi-scale detection helped. (4) Occlusion — pipe partially hidden behind truck/equipment.

Q: How did you evaluate the model?

Primary metric: mAP@0.5. But for safety, recall mattered more than precision — missing a hazard (false negative) is much worse than a false alarm. We tuned confidence threshold lower to prioritize catching all true positives.

Q: How does this apply to Akuret?

Same pattern: detect specific objects in cluttered real-world environments with varying conditions. The detection pipeline, training challenges (dataset quality, environmental variation, similar-looking objects), and deployment considerations are fundamentally the same — different domain, identical engineering.

13. Q&A;: ResNet50 & VGG — Movie Classifier FYP

Q: Walk me through the movie classifier project

Final year project: collected trailers from YouTube, extracted ~200,000 frames, trained multiple CNN architectures (ResNet50, VGG16 in two configs, VGG19) to classify movies as Good/Normal/Bad based on visual content. Good > 6.5 rating, Normal 5.5-6.5, Bad < 5.5.

Q: Why did you use multiple models?

Comparative evaluation — understanding how architecture depth and design choices affect performance. VGG gave a simple baseline. ResNet50 tested whether skip connections beat brute-force depth. The comparison showed ResNet won with 5x fewer parameters.

Q: What was the hardest challenge?

Class imbalance. First labeling scheme produced heavily biased distributions — model just predicted 'Normal' for everything. We re-designed thresholds to 5.5 and 6.5 for balanced classes, and used oversampling and class-weighted loss functions.

Q: Did you use transfer learning?

Yes — necessary given dataset size vs. VGG's 138M parameters. Started with ImageNet pre-trained weights, froze early conv layers, fine-tuned later layers plus new classification head. Training from scratch would have overfit badly.

Q: Which model performed best and why?

ResNet50 — fewer parameters (25.6M vs 138M) meant less overfitting, skip connections enabled effective gradient flow through all 50 layers, and batch normalization in every block stabilized training.

Q: How is this different from what Akuret needs?

This was classification (one label per image). Akuret needs detection (multiple objects with positions). But the CNN backbone is similar — my FYP gave me deep understanding of the feature extraction part; my YOLO project gave me the detection pipeline.

---

14. Q&A;: Data Pipelines — Neighborhoods.com

Q: How did you handle 100+ data sources with different schemas?

Adapter pattern. Common internal data model + per-source adapters for translation. When a source changed format, only that adapter needed updating. Built monitoring/alerting around each integration — caught feed failures before they corrupted downstream analytics.

Q: Tell me about the $40M auditing pipeline

Analyzed a decade of transaction data. Core challenge: record matching — linking MLS listings to county parcel data using property ID, date-proximity, and price-similarity matching. Flagged unreported deal closures where brokerage wasn't recorded. Surfaced $40M+ in recoverable commissions missed by manual auditing.

Q: How does this map to Akuret?

Same pattern. Akuret ingests messy, varied input (shelf scans from different stores/devices), runs it through AI, compares against expected state (planogram), and produces prioritized tasks. My experience with validation layers, data quality from unreliable sources, async processing (Celery + Redis), and reconciliation jobs transfers directly.

15. Q&A;: MLOps, Deployment & Production AI

Q: What's your experience deploying ML models to production?

Full lifecycle: SageMaker for training jobs and endpoints, FastAPI for serving predictions (lightweight, async-native), Docker for containerization, AWS (ECS/EC2) for deployment, GitHub Actions for CI/CD, and monitoring for prediction quality. Model versioning, A/B testing, and alerting on degradation.

Q: Why FastAPI over Django for ML serving?

For inference endpoints: (1) async-native — handles concurrent I/O without blocking, (2) lighter overhead per request for low-latency inference, (3) Pydantic for automatic request/response validation. I use Django for the broader platform — user management, admin, ORM-heavy logic. Right tool for each layer.

Q: How do you know when a model needs retraining?

Three monitoring layers: (1) Data drift — input distribution shifting from training data (new packaging, seasonal changes). (2) Confidence degradation — average prediction confidence declining over time. (3) Feedback loops — if store associates consistently dismiss recommendations, the model is getting something wrong. Triggers should be automated, not manual.

Q: Tell me about your SageMaker experience

Training pipeline: data in S3, SageMaker training jobs pull from there, GPU instances for deep learning, hyperparameter tuning, deploy best model to SageMaker endpoint. Advantage: managed auto-scaling, monitoring, A/B testing. Downside: cost/lock-in — for simpler models I deploy FastAPI on ECS with model from S3 instead.

Q: How do you handle CI/CD for ML projects?

Code and model are separate pipelines. Code: standard CI/CD (lint → test → Docker → ECR → deploy with rolling updates). Model: training triggered by data changes or schedule, artifacts versioned in S3/model registry, deployment via controlled A/B testing. Key principle: code deploys and model deploys are independent and move at different speeds.

Q: Docker and Kubernetes — how have you used them?

Docker for everything — guarantees consistency across environments. For orchestration: used both EKS and ECS. Led a migration from EKS to ECS at ParkGuard — EKS was overkill, added complexity without proportional benefit. For a sub-50 person startup like Akuret, ECS or Docker Compose on EC2 is often the smarter choice.

16. Q&A;: Multi-Tenant SaaS & Enterprise Architecture

Q: How have you built multi-tenant systems?

ParkGuard: multi-tenant enterprise platform with hierarchical partner relationships (partners, subsidiaries, admins). Designed RBAC with MFA, API-key auth, IP whitelisting, audit logging. Every API endpoint enforced tenant context — zero cross-tenant data leakage. Same pattern applies to Akuret: each retail chain = tenant, each store = sub-tenant, corporate = roll-up view.

Q: How do you ensure data isolation between tenants?

Defense in depth — three layers: (1) Database: tenant-scoped queries everywhere, enforced at ORM level. (2) API: middleware extracts tenant context from auth token, injects into every handler. (3) Infrastructure: tenant-prefixed S3 paths with IAM policies. A bug in one layer can't expose another tenant's data.

---

17. Q&A;: General AI/ML Concepts

Q: Supervised vs unsupervised learning?

Supervised: labeled examples (image = Coke, image = empty). Both YOLO and FYP were supervised. Unsupervised: no labels — model finds patterns (clustering, anomaly detection). Semi-supervised: label small subset, use model predictions to pseudo-label the rest.

Q: Model works in testing but fails in production?

Almost always data distribution mismatch. Steps: compare training vs production distributions, check for data leakage in test set, add production-representative data to training, set up monitoring to catch this faster next time.

Q: Explain overfitting for shelf scanning

Train only on well-lit, perfectly stocked shelves in one chain → great test performance. Deploy in different chain with different lighting, shelf heights, packaging → performance drops. Model memorized specific patterns instead of learning general features. Fix: diverse training data, augmentation, dropout, transfer learning.

Q: Model accuracy vs inference speed trade-off?

Bigger = more accurate but slower. Store associates won't wait 30 seconds. Use smaller YOLO variant (YOLOv5s), model distillation (compress large model into small one), or quantization (32-bit → 8-bit weights, ~2x faster).

Q: What are embeddings and how would they help?

Dense vectors capturing semantic meaning — similar things are close in vector space. For retail: encode product images into embeddings, then identify new products by nearest neighbor search — no retraining needed. I've built embedding-based systems with pgvector for semantic job matching at AI Career Ops. Same pattern applies to product matching.

18. Smart Questions to Ask Kevin

Asking sharp questions matters as much as answering well. These show you've thought beyond the surface level.

On the CV pipeline:

> *"How does the model handle new products it hasn't seen before — like when a store introduces a new SKU or changes its planogram?"*

On architecture:

> *"Are you using a single-stage detector like YOLO, or did the shelf environment need a two-stage approach for accuracy? And how do you handle fine-grained SKU distinction — similar products with different flavors?"*

On scale:

> *"What does the integration architecture look like for connecting with different POS and ERP systems across retail chains? Is it adapter-based, or is there a standard they all conform to?"*

On team:

> *"How large is the engineering team today, and what's the split between ML/CV engineers and platform/infrastructure engineers?"*

On challenges:

> *"What's the biggest technical challenge you're facing right now — is it model accuracy, scale, integration complexity, or something else?"*

Personal (shows research):

> *"You've been building enterprise software for 20+ years across manufacturing and now retail — what's been the most surprising difference in building for grocery retail specifically?"*